

```

"""
ARKHEIA-CPS – Perfection Law Module
Analyzes secured interest perfection timing
for auto loans.
Detects illegal patterns, predatory timing,
and procedural violations.

Legal basis: UCC Article 9 + state-specific
title lien notation statutes.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from datetime import date
from decimal import Decimal
from typing import List, Optional

from app.schemas.auto import AutoContractIn
from app.schemas.common import Alert,
AlertSeverity, PEMEvaluation
from app.utils.statutes import
get_auto_perfection_rules

# — Timing thresholds (days after
purchase_date) —————
PERF_EXPECTED_MIN   = 10
PERF_EXPECTED_MAX   = 30
PERF_MODERATE_MAX    = 60
PERF_HIGH_MAX        = 90
# > 90 days = severe deviation

```

```

@dataclass
class PerfectionResult:
    perfection_days: Optional[int]
    = None    # days from purchase to
    perfection
    risk_score: Decimal
    = Decimal("0")
    alerts: List[Alert]
    = field(default_factory=list)
    evaluations: List[PEMEvaluation]
    = field(default_factory=list)
    illegal_patterns: List[str]
    = field(default_factory=list)
    predatory_flags: List[str]
    = field(default_factory=list)

class PerfectionLawService:
    """
    Analyzes lien perfection timing and
    sequencing for auto loan contracts.
    Produces risk scores, violation alerts,
    and plain-language explanations.
    """

    def analyze(self, payload:
AutoContractIn) -> PerfectionResult:
        result = PerfectionResult()

        purchase = payload.purchase_date
        perfection =
payload.perfection_date

```

```

        repo      =
payload.repossession_date
        notice_def =
payload.notice_of_default_date
        right_cure =
payload.right_to_cure_notice_date
        state      =
payload.jurisdiction_state

        # Load state-specific rules
        rules =
get_auto_perfection_rules(state)

        # — 1. Perfection timing


---


        if purchase and perfection:
            days = (perfection -
purchase).days
            result.perfection_days = days
            result.evaluations +=
self._eval_timing(days, purchase,
perfection, rules)
        elif purchase and not perfection:
            result.alerts.append(Alert(
                code="PERF_MISSING",

severity=AlertSeverity.HIGH,
                message=(
                    "No perfection date is
recorded. The lender may not have properly
"

                    "perfected their
security interest, which affects the
legality of "

```

```

        "any repossession or
enforcement action."
    ),

    statute=rules.get("perfection_statute"),
    ))

result.illegal_patterns.append("No
perfection date recorded")

result.evaluations.append(PMEvaluation(

pattern_family="PERFECTION",
pattern_key="PERF_DATE_PRESENT",
    status="FAIL",
    explanation="Perfection
date is missing. Cannot verify lien was
properly established."
    ))

# — 2. Perfection after
repossession —————
    if perfection and repo:
        if perfection > repo:
            result.alerts.append(Alert(
                code="PERF_AFTER_REPO",

severity=AlertSeverity.EXTREME,
                message=(
                    f"The lien was
perfected on {perfection} AFTER the vehicle
was "

                    f"reposessed on
{repo}. Repossession before perfection is

```

```

an illegal "
                f"enforcement
action under UCC Article 9. The lender had
no perfected "
                f"security interest
at the time of repossession."
            ),

statute=rules.get("repo_statute"),
        ))

result.illegal_patterns.append("PERF_AFTER_
REPO: Lien perfected after repossession")

result.evaluations.append(PMEvaluation(

pattern_family="PERFECTION",
pattern_key="PERF_SEQUENCE",
                status="FAIL",
                explanation="ILLEGAL:
Perfection occurred after repossession.
Repossession was unlawful."
            ))

        elif perfection == repo:
            result.alerts.append(Alert(
                code="PERF_SAME_DAY",

severity=AlertSeverity.EXTREME,
                message=(
                    f"The lien was
perfected and the vehicle was repossessed
on the same "
                    f"date ({repo}).

```

This is a predatory pattern indicating the lender "

```
                f"perfected  
specifically to immediately repossess – a  
bad-faith enforcement action."  
            ),
```

```
statute=rules.get("repo_statute"),  
    ))
```

```
result.predatory_flags.append("PERF_SAME_DA  
Y: Perfection and repossession on same  
date")
```

```
result.evaluations.append(PMEvaluation(
```

```
    pattern_family="PERFECTION",  
    pattern_key="PERF_SAME_DAY",  
        status="FAIL",  
        explanation="PREDATORY:  
Lien perfected and vehicle repossessed on  
the same day."  
    ))
```

— 3. Repossession before notice
of default —————

```
    if repo and notice_def:  
        if repo < notice_def:  
            result.alerts.append(Alert(  
                code="REPO_NO_NOTICE",  
  
severity=AlertSeverity.EXTREME,  
                message=(  
                    f"The vehicle was
```

```

repossessed on {repo} before the notice of
"
                f"default was sent
on {notice_def}. Repossession without
proper "
                f"default notice is
illegal in most states."
            ),

statute=rules.get("notice_statute"),
        ))

result.illegal_patterns.append("REPO_NO_NOT
ICE: Repossession before notice of
default")

result.evaluations.append(PMEvaluation(

pattern_family="PERFECTION",
pattern_key="REPO_DEFAULT_SEQUENCE",
                status="FAIL",
                explanation="ILLEGAL:
Vehicle repossessed before notice of
default was issued."
            ))

# — 4. Repossession before right-
to-cure —————
    if repo and right_cure:
        if repo < right_cure:
            result.alerts.append(Alert(

code="REPO_BEFORE_CURE",

```

```

severity=AlertSeverity.EXTREME,
    message=(
        f"The vehicle was
repossessed on {repo} before the right-to-
cure "
        f"notice date of
{right_cure}. Georgia O.C.G.A. § 10-1-36
requires "
        f"that consumers
receive a right-to-cure notice before
repossession. "
        f"This repossession
was unlawful."
    ),

statute=rules.get("cure_statute"),
))

result.illegal_patterns.append("REPO_BEFORE
_CURE: Repossession before right-to-cure
notice")

result.evaluations.append(PMEvaluation(

pattern_family="PERFECTION",
pattern_key="REPO_CURE_SEQUENCE",
    status="FAIL",
    explanation="ILLEGAL:
Vehicle repossessed before right-to-cure
notice was issued."
))

# Check if cure period
(typically 10-20 days) was honored

```



```

        elif repo and right_cure:
            cure_days_given = (repo -
right_cure).days
            min_cure =
rules.get("min_cure_days", 10)
            if cure_days_given <
min_cure:

result.alerts.append(Alert(

code="REPO_CURE_TOO_SOON",

severity=AlertSeverity.HIGH,
                message=(
                    f"Only
{cure_days_given} day(s) elapsed between
the right-to-cure "
                    f"notice and
repossession. State law requires at least
{min_cure} days. "
                    f"The cure
period was not adequately honored."
                ),

statute=rules.get("cure_statute"),
            ))

result.illegal_patterns.append(f"REPO_CURE_
T00_SOON: Only {cure_days_given} days cure
period given")

# — 5. Title / lien notation
check —————
    if payload.title_issued_date and

```

```

payload.title_lien_notation_date:
    lien_to_title =
(payload.title_lien_notation_date -
payload.title_issued_date).days
    if lien_to_title > 30:
        result.alerts.append(Alert(

code="LIEN_NOTATION_LATE",

severity=AlertSeverity.MODERATE,
    message=(
        f"Lien notation on
title occurred {lien_to_title} days after
title was issued. "
        f"Delayed lien
notation may create a gap in perfection."
    ),
))

result.evaluations.append(PEMEvaluation(

pattern_family="PERFECTION",
pattern_key="LIEN_NOTATION_TIMING",

expected_max=Decimal("30"),
actual_value=Decimal(str(lien_to_title)),
    status="WARN",
    explanation=f"Lien
notation on title took {lien_to_title} days
– above 30-day guideline."
))

# — 6. Compute risk score

```

```

        result.risk_score =
self._compute_risk_score(result)

        return result

# — Timing evaluation
—

    def _eval_timing(self, days: int,
purchase: date, perfection: date, rules:
dict) -> List[PEMEvaluation]:
        evals = []

        expected_days =
rules.get("expected_max_days",
PERF_EXPECTED_MAX)

        if days < 0:
            evals.append(PEMEvaluation(

pattern_family="PERFECTION",
pattern_key="PERF_BEFORE_PURCHASE",

actual_value=Decimal(str(days)),
status="FAIL",
                explanation=f"ANOMALY:
Perfection date ({perfection}) is before
purchase date ({purchase})."
            ))
        elif days <= expected_days:
            evals.append(PEMEvaluation(

pattern_family="PERFECTION",

```

```

pattern_key="PERF_TIMING",

expected_min=Decimal(str(PERF_EXPECTED_MIN)
),

expected_max=Decimal(str(expected_days)),

actual_value=Decimal(str(days)),
status="OK",
        explanation=f"Lien
perfected {days} days after purchase –
within expected {PERF_EXPECTED_MIN}-
{expected_days} day window."
    ))
    elif days <= PERF_MODERATE_MAX:
        evals.append(PMEvaluation(

pattern_family="PERFECTION",
pattern_key="PERF_TIMING",

expected_max=Decimal(str(expected_days)),

actual_value=Decimal(str(days)),
status="WARN",
        explanation=(
            f"Lien perfected {days}
days after purchase. Expected:
{PERF_EXPECTED_MIN}-{expected_days} days. "
            f"Moderate delay may
indicate administrative issues."
        )
    ))
    elif days <= PERF_HIGH_MAX:
        evals.append(PMEvaluation(

```

```

pattern_family="PERFECTION",
pattern_key="PERF_TIMING",

expected_max=Decimal(str(expected_days)),

actual_value=Decimal(str(days)),
status="FAIL",
    explanation=(
        f"Lien perfected {days}
days after purchase – high deviation. "
        f"Expected:
{PERF_EXPECTED_MIN}-{expected_days} days. "
        f"This delay may affect
the enforceability of the lien."
    )
))
else:
    evals.append(PMEvaluation(

pattern_family="PERFECTION",
pattern_key="PERF_TIMING_SEVERE",

expected_max=Decimal(str(expected_days)),

actual_value=Decimal(str(days)),
status="FAIL",
    explanation=(
        f"Lien perfected {days}
days after purchase – SEVERE deviation. "
        f"Typical perfection:
{PERF_EXPECTED_MIN}-{expected_days} days. "
        f"This severe delay may
affect the legality of repossession and

```

```

enforcement."
        )
    ))

    return evals

# — Risk scoring

```

```

    def _compute_risk_score(self, result:
PerfectionResult) -> Decimal:
        score = Decimal("0")

        # Timing-based deduction
        days = result.perfection_days
        if days is not None:
            if days > PERF_HIGH_MAX:
                score += Decimal("60")
            elif days > PERF_MODERATE_MAX:
                score += Decimal("40")
            elif days > PERF_EXPECTED_MAX:
                score += Decimal("20")

        # Illegal pattern deductions
        score +=
Decimal(str(len(result.illegal_patterns) *
25))

        # Predatory flag deductions
        score +=
Decimal(str(len(result.predatory_flags) *
20))

```

```
        # Missing perfection date
        if result.perfection_days is None
and any(a.code == "PERF_MISSING" for a in
result.alerts):
            score += Decimal("40")

    return min(score, Decimal("100"))
```